In-Class Exercise/Demo

In-Class Demo

- We'll get some practice on React.
- First, create a new repository **m5-w2-d1-exercise** in your remote Github and clone this in the WESTCLIFF folder.
- Next head over to GAP and download m5-w2-d1-demo.zip folder, unzip and drop the content into the clone folder.
- Launch VS Code and navigate to the clone folder.
- Open demo1.js
- We'll create our first form.
- Within the render method we will create our form HTML.
- But first let's add a handleChange method.

In-Class Demo #1

- In demo1.js after constructor let's add

```
handleChange(event) {
    this.setState({value: event.target.value});
  }
```

Both of these methods will execute when something is changed or a form is submitted.

- Now we'll add the handleSubmit method

```
handleSubmit(event) {
    alert('A name was submitted: ' + this.state.value);
    event.preventDefault();
  }
```

In-Class Demo #1

- Now lets update our render to render the form

```
render() {
     return (
       <form onSubmit={this.handleSubmit}>
         <label>
           Name:
           <input type="text" value={this.state.value}
onChange={this.handleChange} />
         </label>
         <input type="submit" value="Submit" />
       </form>
     );
   }
```

In-Class Demo #1

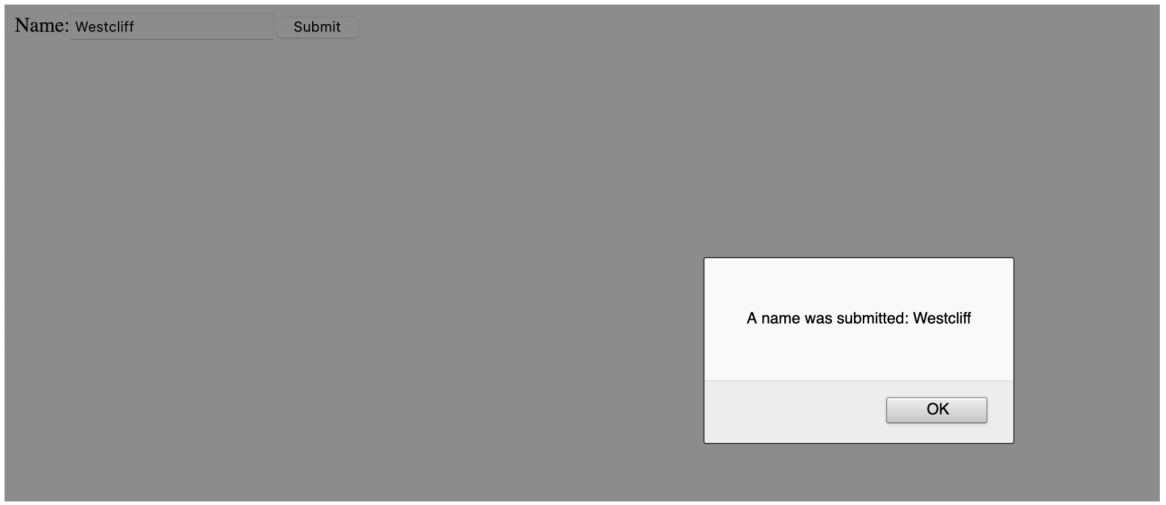
- This completes our methods and form. Now we need to update our constructor to make sure it can handle both methods. We will using this keyword.

```
constructor(props) {
     super(props);
     this.state = {value: ''};
     this.handleChange = this.handleChange.bind(this);
     this.handleSubmit = this.handleSubmit.bind(this);
   }
```

In-Class Demo #1

- Open using Live server

Enter Value and
click submit



In-Class Demo #2

- Open demo2.js
- Let’s create our first Multiple fields.
- First We will update our handle methods

```
handleInputChange(event) {
     const target = event.target;
     const value = target.type === 'checkbox' ?
target.checked : target.value;
     const name = target.name;
      this.setState({
        [name]: value
     });
   }
```

In-Class Demo #2

- Next let's update our form to add multiple fields.

```
<form>
        <label>
          Is going:
          <input
            name="isGoing"
            type="checkbox"
            checked={this.state.isGoing}
            onChange={this.handleInputChange} />
        </label>
        <br />
        <label>
          Number of guests:
          <input
            name="numberOfGuests"
            type="number"
            value={this.state.numberOfGuests}
            onChange={this.handleInputChange} />
        </label>
      </form>
```

In-Class Demo #2

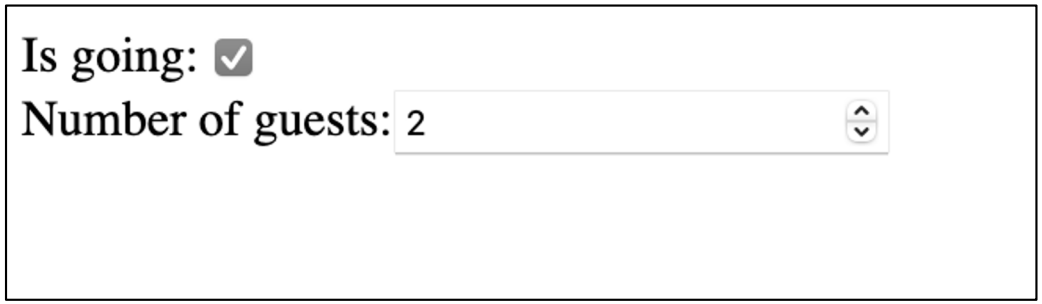
- Next let's update our constructor to add the state.

```
this.state = {
        isGoing: true,
        numberOfGuests: 2
 };
```


We are adding default state for both fields

In-Class Demo #2

- Let’s open in live server



In-Class Demo #3

- Next we will practice same forms using uncontrolled components
- Open demo3.js
- We will create 1 handle and create a ref

```
constructor(props) {
     super(props);
     this.handleSubmit = this.handleSubmit.bind(this);
     this.input = React.createRef();
   }
    handleSubmit(event) {
     alert('A name was submitted: ' + this.input.current.value);
     event.preventDefault();
   }
```

In-Class Demo #3

- Next let's add code to let our input know the ref.

```
<input type="text" ref={this.input} />
```

- Open the page in browser. The behavior will be same.

In-Class Demo #4

- File uploads are uncontrolled components only. Now we will practice a file upload.
- Open demo4.js
- First let's add code to handleSubmit

```
event.preventDefault();
      alert(
        `Selected file -
${this.fileInput.current.files[0].name}`
      );
```

In-Class Demo #4

- Next lets update the constructor.

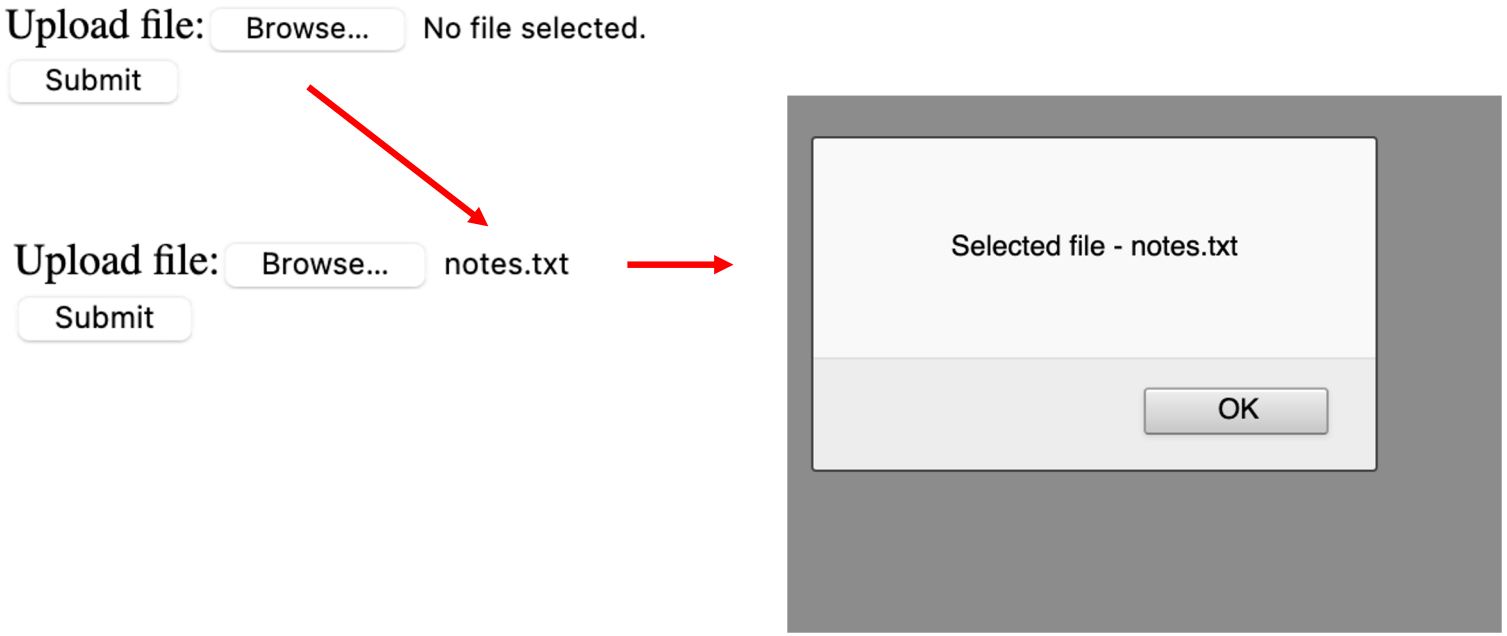
```
this.handleSubmit = this.handleSubmit.bind(this);
this.fileInput = React.createRef();
```

- Next add the Input

```
<label>
       Upload file:
       <input type="file" ref={this.fileInput} />
</label>
```

In-Class Demo #4

- Open in Live server. Upload a file and you will see that the popup remembers it.



In-Class Exercise Demo

Due:

Today 10.30PM PT

Submission:

Post Github URL Link on GAP Week 2 Day 1 Exercise